pyALDIC User Guide

Augmented Lagrangian Digital Image Correlation
Desktop GUI Walkthrough

pyALDIC Project

August 25, 2026

Abstract

This guide walks a new user through a complete pyALDIC analysis, from first launching the GUI to exporting results. Each section maps directly to a panel or action in the application so you can follow along with the real interface open.

If you already know pyALDIC and just need a refresher on one step, jump to the relevant section via the table of contents.

Contents

1 Overview **4**
- 1.1 Three-column layout . . . 4
- 1.2 Typical session times . . . 5

2 Installing and launching **5**
- 2.1 Installation . . . 5
- 2.2 Launching the GUI . . . 6
- 2.3 What you should see first . . . 6

3 Loading images **7**
- 3.1 Dropping or browsing a folder . . . 7
- 3.2 The image list . . . 7
- 3.3 Navigating frames . . . 8

4 Choosing the Workflow Type **8**
- 4.1 Tracking Mode . . . 8
- 4.2 Solver . . . 8
- 4.3 Reference Update (incremental only) . . . 8
- 4.4 Which combination to pick? . . . 9

5 Drawing a Region of Interest **9**
- 5.1 Live hint . . . 9
- 5.2 The Region of Interest toolbar . . . 9
- 5.3 Drawing shapes . . . 11
- 5.4 Per-frame editing . . . 11

6 DIC Parameters **11**
6.1 Subset Size . . . 11
6.2 Subset Step . . . 12
6.3 Search Range . . . 12
6.4 Mesh Refinement . . . 13

7 Advanced parameters **13**
7.1 Initial Guess . . . 13
7.2 Starting Points workflow . . . 14
7.2.1 Multi-seed tolerance . . . 14
7.2.2 Reference-frame switches in Starting Points mode . . . 15
7.3 Auto-expand search on clipped peaks . . . 15
7.4 Mesh appearance . . . 15

8 Canvas interactions **15**
8.1 Zoom and pan . . . 16
8.2 Frame navigation . . . 16
8.3 Show on deformed frame . . . 16

9 Running the DIC **16**
9.1 The Run button . . . 16
9.2 Cancelling . . . 16
9.3 Progress display . . . 17
9.4 Fatal errors . . . 17
9.5 Completion . . . 17

10 Viewing results **17**
10.1 FIELD section . . . 17
10.2 VISUALIZATION section . . . 17
10.3 PHYSICAL UNITS section . . . 18
10.4 Console log . . . 18

11 Strain post-processing **19**
11.1 Strain Parameters panel . . . 19
11.2 Compute Strain button . . . 21
11.3 Strain field selector . . . 21
11.4 Strain vs main window . . . 22

12 Exporting **22**
12.1 The Export dialog . . . 22
12.2 Destination folder . . . 24
12.3 Running an export . . . 24

13 Sessions — save and resume **25**
13.1 File format . . . 25
13.2 What is saved . . . 25
13.3 What is not saved . . . 25
13.4 Menu . . . 25
13.5 Including results . . . 26

13.6 Opening a session whose images moved . . . 26
13.7 Corrupt or future-version files . . . 26
13.8 Legacy sessions from pyALDIC 0.5 and earlier . . . 26

14 Troubleshooting **27**
14.1 “Cannot start analysis” . . . 27
14.2 “DIC analysis failed” . . . 27
14.3 Starting Points errors . . . 27
14.4 The very first analysis takes far longer than the rest . . . 28
14.5 Slow run . . . 28
14.6 Poor results in specific regions . . . 28
14.7 Strain map is blank or export is all NaN . . . 28
14.8 Large rotation gives crazy strain values . . . 29
14.9 Refine brush grayed out . . . 29
14.10 Session file won’t open . . . 29
14.11 Windows bundle: “Windows protected your PC” . . . 29
14.12 Windows bundle: nothing happens on double-click . . . 29

1 Overview

pyALDIC computes full-field displacement and strain from a sequence of grayscale images of a deforming specimen. The typical workflow, from left to right through the GUI, is:

1. **Load images** (a folder of frames, first frame = reference).
2. **Choose Workflow Type** — tracking mode, solver, and reference-update policy. These choices are made *before* drawing a Region of Interest because they determine which frames need one.
3. **Draw a Region of Interest** on the required frame(s).
4. **Set DIC parameters** — subset size, node spacing, search range, mesh refinement.
5. **Run the analysis.** Progress, Cancel, and console log are on the right panel.
6. **View displacement results** live in the canvas.
7. **Compute strain** in the Strain Post-Processing window (opens automatically when the run finishes).
8. **Export** numerical data, images, or animations.
9. **Save the session** (File → Save Session) — one `.aldic` file keeps parameters, Regions of Interest, view state, and the computed results, so you can reopen the project later without recomputing.

Note

Video tutorials. A full end-to-end walkthrough of this workflow is available on YouTube (https://www.youtube.com/watch?v=aLqDyM3tQII, English; https://www.youtube.com/watch?v=27TtJEMefNA, Chinese) and on Bilibili (https://www.bilibili.com/video/BV1dBjm6zEnQ, P1 Chinese / P2 English).

Note

pyALDIC does **not** require you to pre-compute calibration, stereo matching, or any 3D reconstruction. It is strictly 2D DIC (one camera, planar specimen). If the specimen moves out of plane or the illumination changes significantly between frames, results will degrade and the GUI will warn you.

1.1 Three-column layout

The main window has three columns:

Left sidebar
: Images panel, *Workflow Type* section, *Region of Interest* section, *Parameters* section, and a collapsible *Advanced* section.

Centre canvas
: Zoom / pan viewport showing the current frame with overlays (ROI, mesh, displacement field).

Right sidebar
: Run button, progress bar, field selector (disp_u, disp_v, etc.), visualization controls (colormap, range, opacity), physical units, and console log.

1.2 Typical session times

On a modern laptop (8-core CPU, 16 GB RAM):

Image size	Grid spacing	Time per frame
256×256	step 8	$\sim$ 0.2 s
512×512	step 8	$\sim$ 1 s
1024×1024	step 4	$\sim$ 3 s

Times scale linearly with the number of mesh nodes. The *Local DIC* solver is roughly 3× faster than *AL-DIC*.

2 Installing and launching

2.1 Installation

Four ways to install, ordered by convenience. Option A needs nothing installed at all; B, C and D require Python 3.10 or newer.

Option A: Windows bundle (no Python needed)

The whole application in a folder you unzip and run. Nothing to install, no administrator rights, no interpreter.

1. Go to https://github.com/zachtong/pyALDIC/releases/latest and download `pyALDIC-<version>-win`
2. Unzip it somewhere you can write — your Desktop or Documents folder is fine.
3. Open the folder and double-click `pyALDIC.exe`.

Windows 10 (version 1703 or later) and Windows 11, 64-bit. About 500 MB unzipped. Keep the folder together: the executable needs the files beside it, so move or copy the whole folder rather than the `.exe` alone.

Note

The bundle is not code-signed, so on first launch Windows shows *"Windows protected your PC"*. Choose **More info**, then **Run anyway**. This is SmartScreen reporting that the file is not yet widely downloaded, not a virus detection.

Beside `pyALDIC.exe` sits `pyALDIC-console.exe`: the same application with a console window attached, useful when you need to see an error message that the normal build has nowhere to print.

Option B: From PyPI (recommended if you have Python)

```
pip install al-dic
```

This pulls the latest released wheel and every runtime dependency in one step.

Option C: From a GitHub Release wheel

Useful when you cannot reach PyPI (corporate firewall, air-gapped network) or when you want to install a specific past version whose artefacts are still attached to its release page.

1. Go to https://github.com/zachtong/pyALDIC/releases.
2. Expand *Assets* on the release you want; download the `al_dic-<version>-py3-none-any.whl` file.
3. Install locally:

```
pip install ./al_dic-<version>-py3-none-any.whl
```

Dependencies are still fetched from PyPI at install time, so this method only bypasses PyPI for the `al-dic` package itself.

Option D: From source (development)

Editable install that lets you modify the code and see the effect without re-installing.

```
git clone https://github.com/zachtong/pyALDIC.git
cd pyALDIC
pip install -e ".[dev]"
```

The `.[dev]` extras include `pytest` and `pytest-xdist` for running the test suite.

2.2 Launching the GUI

From a Python install (options B–D):

```
al-dic
# or equivalently
python -m al_dic
```

From the Windows bundle (option A), double-click `pyALDIC.exe` inside the unzipped folder.

The window will not shrink below 1120×760. You can resize it freely above that; the three columns keep their proportions.

2.3 What you should see first

The first time the window opens:

- The left image list is empty and shows a drop zone that reads *"Drop image folder or Browse"*.
- The centre canvas shows an empty dark background.
- The right sidebar shows a disabled **Run DIC Analysis** button (you cannot run without images and a Region of Interest).

Note

Shortly after the window appears, the console log reads *"Preparing compute kernels in the background"*. The solver is compiled to machine code the first time it runs and the result is cached, so this happens once per installation — pyALDIC starts it early so that it overlaps loading images and drawing a Region of Interest rather than stalling your first analysis. The interface may feel slightly less responsive for those few tens of seconds. A second message reports when it is done. Installing a new version invalidates the cache, so it happens again after an upgrade.

Tip

All user-visible text uses the full phrase *Region of Interest* instead of the DIC jargon "ROI". The 50-pixel Region column in the image list is abbreviated to "Region" to fit, with Need/Edit/Add button labels inside.

3 Loading images

3.1 Dropping or browsing a folder

Click the drop zone or drag a folder of image files onto it. Every image in the folder is loaded in natural sort order. The first image is always treated as the reference frame (frame 1 in the UI, index 0 internally).

Supported formats: `.tif`, `.tiff`, `.png`, `.bmp`, `.jpg`, `.jpeg`. Grayscale is assumed; RGB inputs are converted to luminance on load.

Warning

All images in the folder must have the same dimensions. If they don't, loading stops and an error dialog tells you which image is the first mismatch.

Note

Large image sets. Frames are streamed from disk on demand rather than pre-loaded into RAM, and internal caches are bounded, so memory use stays roughly flat with the number of frames — hundreds of 4K images are fine on a 16 GB machine. Keep the folder on a fast local drive for the smoothest scrubbing.

3.2 The image list

After loading, the left sidebar shows three columns:

Column	Width	Content
#	fixed	1-based frame number
Filename	flex	file name without extension
Region	50 px	Need / Edit / Add button

The *Region* column button changes its label based on state:

Need This frame is a reference frame (per the current workflow type) and does not yet have a Region of Interest.

Edit This frame already has a Region of Interest; click to modify it.

Add This frame is *not* a required reference frame, but you can optionally add one.

Clicking any row's button activates Region of Interest editing for that specific frame; the canvas switches to show that frame's reference and the *EDITING REGION OF INTEREST FOR FRAME xx* banner appears.

3.3 Navigating frames

The frame slider at the bottom of the canvas scrubs through the sequence. Clicking a row in the image list also jumps to that frame. Keyboard: left/right arrow keys move by one frame when the canvas has focus.

4 Choosing the Workflow Type

The *Workflow Type* section is the first choice you make after loading images. It determines:

- How each frame is compared against a reference,
- Which solver (Local DIC or AL-DIC) runs,
- How often the reference frame refreshes (incremental only),
- **and therefore, which frames need a Region of Interest.**

Tip

Get this right before drawing Regions of Interest. The Region of Interest section below shows a live hint that updates whenever you change the workflow type, telling you exactly which 1-based frame numbers require a region.

4.1 Tracking Mode

Accumulative
: Every frame is compared directly against frame 1. Best for small, monotonic deformation (strain $< 5\%$, rotation $< 2°$). Fast, but fails for large cumulative motion because the reference and current subsets become visually too different.

Incremental
: Each frame is compared against a rolling reference frame. Required for rotations $> 5°$ or large cumulative strain. Displacements from consecutive segments are composed into cumulative output fields. Slightly slower than accumulative but handles arbitrary total deformation.

4.2 Solver

Local DIC
: Each subset is solved independently with IC-GN. Fast, preserves sharp local features, but sensitive to noise. Best for high- quality images and small deformations.

AL-DIC
: Augmented Lagrangian solver that couples local IC-GN with a global FEM regularizer. Smoother results, better strain accuracy, more robust in noisy or high-gradient regions. $\sim 3\times$ slower than Local DIC. Default.

When AL-DIC is selected, an indented *ADMM Iterations* spinbox appears (1–10, default 3). Higher is more regularized but slower.

4.3 Reference Update (incremental only)

Visible only when *Tracking Mode* = Incremental. Controls when the rolling reference frame refreshes:

Every Frame
Reference resets every frame. Most robust for large deformation (smallest per-step ratio). A Region of Interest is only needed on frame 1 — pyALDIC warps it forward automatically.

Every N Frames
Reference resets every N frames. You need a Region of Interest on frame 1, $N+1$, $2N+1$, ... The live hint in the Region of Interest section lists the required frames.

Custom Frames
You type a comma-separated list of reference frames, e.g. `1, 6, 12, 24`. Frame 1 is always included.

4.4 Which combination to pick?

Experiment	Tracking	Reference Update
Soft material, $< 2\%$ strain	Accumulative	—
Soft material, 2–30% strain	Incremental	Every Frame
Rigid rotation, $> 5°$	Incremental	Every Frame
Dynamic event / impact	Incremental	Every Frame
Quasi-static with known load steps	Incremental	Every N or Custom (align N with steps)

Note
When in doubt, use *Incremental + Every Frame*. It is slightly slower than accumulative but handles every deformation magnitude well, and only needs one Region of Interest drawn on frame 1.

5 Drawing a Region of Interest

5.1 Live hint

The top of the *Region of Interest* section is a blue-accent info box that updates live as you change the workflow type. It tells you exactly which frames need a region:

- *Accumulative* → "Only frame 1 needs a Region of Interest. All later frames are compared against it directly."
- *Incremental + Every Frame* → "Frame 1 needs a Region of Interest. It is automatically warped forward to each later frame."
- *Incremental + Every N Frames* → "Draw a Region of Interest on frames: 1, 6, 11, 16, ..."
- *Incremental + Custom Frames* → Lists your custom reference frame indices.

5.2 The Region of Interest toolbar

Three rows of buttons, from top to bottom:

Row 1 — Shape operations

+ Add ▼
Opens a popup menu: Polygon / Rectangle / Circle / Circle (3-point). Pick one, then draw on the canvas. The drawn region is added (OR'd) to the Region of Interest mask.

Cut ▼

Same menu as Add, but the drawn region is subtracted (AND-NOT) from the Region of Interest mask. Use this to cut holes around features you don't want to track (bubbles, voids, damaged areas).

+ Refine ▼

Paint brush for marking mesh-refinement zones (does *not* change the Region of Interest — it marks areas where the quadtree mesh should be subdivided for finer resolution).

Warning

The Refine brush is gated to frame 1 only. On any other frame the button renders with a dashed border and muted colours; hovering shows the explanation. This is because the brush paints in frame-1 pixel coordinates and is automatically warped forward — painting on a deformed frame would place refinement zones in the wrong material points.

Row 2 — Import / Batch Import

Import

Load a grayscale / binary image and use it as the Region of Interest for the currently-edited frame. Any pixel above zero counts as "inside".

Batch Import

Opens a dialog to import several mask files at once and assign them to specific frames. Useful when you have pre-segmented masks from e.g. SAM, thresholding, or a dedicated segmentation tool.

The dialog has three panels:

- **Available Masks** — files scanned from a folder you pick with *Browse*. Masks whose on-disk size does not match the loaded images are greyed out (the tooltip explains the mismatch) and excluded from every assignment path. Both 0/1 and 0/255 mask encodings are supported.
- **Frame Assignments** — only the frames the pipeline will actually consume a mask for are shown: frame 1 in accumulative mode, and every reference frame (per your Reference Update setting) in incremental mode. Non-reference frames are hidden because the solver never reads their masks.
- **Preview** — live image + mask overlay for the currently focused frame. Three view modes (image only, image + mask, mask only), an α slider 0–100%, and four overlay colours. Scroll to zoom (cursor anchored), left-drag to pan.

Assignment strategies:

- *Auto-Match by Name* — extract the last number in each mask filename and match to that frame index.
- *Assign Sequential* — pair the sorted mask list with the sorted required-frame list 1:1.
- *Assign Selected* — manual pairing. Selecting one mask plus several frames broadcasts the same mask to every selected frame (1:N). Selecting several masks plus one frame is rejected because the relationship is one-mask-per-frame only.

Row 3 — Save / Invert / Clear

Save Writes the current frame's Region of Interest mask to a PNG.

Invert
Flips inside/outside on the current frame.

Clear
Empties the current frame's Region of Interest mask. Does not delete other frames' regions.

5.3 Drawing shapes

After selecting a shape from Add or Cut:

Polygon
Click to place vertices, double-click or press Enter to close. Right-click to cancel.

Rectangle
Click-drag to define the bounding box.

Circle
Click the centre, drag outward to set the radius, release to commit.

Circle (3-point)
Click three points on the circle's edge; the circle passing through them (the circumcircle) is fitted automatically. After the first two points a preview circle follows the cursor; the third click commits. Useful when you can see points on a circular feature's edge but the centre is not obvious. If the three points are (nearly) collinear no circle can be formed and the action is cancelled with a warning, so spread the points around the edge.

While drawing, the toolbar button for the active mode highlights. The canvas cursor changes to a crosshair. Click *Esc* or the already-highlighted button again to cancel.

5.4 Per-frame editing

Click the Region column button for any frame to jump to editing that frame's region. The canvas shows the selected frame's image (in frame-1 coordinates for accumulative mode, or whichever ref frame you're editing for incremental). A banner overlays the top of the canvas:

```
EDITING REGION OF INTEREST FOR FRAME 07
```

Changes are committed as soon as you finish a shape operation.

6 DIC Parameters

The *Parameters* section of the left sidebar holds the subset- and mesh-related choices. Change any of these and the downstream pipeline will recompute when you press Run.

6.1 Subset Size

The IC-GN correlation window, in pixels. Displayed as an odd number (e.g. 41), stored internally as the even half-width (40).

Value	When to use
21–31	Fine / dense speckle, small strain. Best spatial resolution.
41 (default)	General speckle patterns with $\sim$ 3–5 px particles.
51–81	Noisy images, coarse speckle, or low-contrast regions. Lower spatial resolution.

Larger subsets average over more pixels, which improves signal-to- noise but smooths out local features. Rule of thumb: each subset should contain 5–10 speckle particles.

6.2 Subset Step

Node spacing in pixels, restricted to powers of two: `4, 8, 16, 32, 64`. Smaller step means more mesh nodes and a denser displacement field.

Value	Trade-off
4	Densest grid, very slow. Use only for $\leq 512^2$ images.
8 (default for 512^2)	Good balance.
16 (default for 1024^2)	Standard choice for large images.
32–64	Very fast, coarse. Only for quick previews.

Tip

Total mesh nodes grow as (ROI area)/(step)^2. Halving the step quadruples the node count and roughly quadruples runtime. Start coarse, iterate on Region of Interest and parameters, then refine the step only when you are happy with the setup.

6.3 Search Range

Maximum per-frame displacement (in pixels) that the FFT initial search can detect. If the real displacement exceeds this, the FFT returns a clipped peak and pyALDIC auto-expands the search radius (2× each retry up to image half-size) — but this costs time.

Value	When to use
8–20 (default 20)	Typical quasi-static experiments.
40–60	Larger translations, moderate rotations in incremental mode.
80+	Use only if you know the motion is huge. Diagnose before setting high: a large search range also means more chance of picking wrong NCC peaks in repetitive textures.

Warning

If you see `FFT search: N nodes have peaks at search boundary` in the console log, the initial search range is too small. Either raise it manually or let Auto-expand (in Advanced, on by default) handle it.

6.4 Mesh Refinement

Two independent checkboxes below the search range control quadtree refinement of the mesh:

Refine Inner Boundary
: Subdivide mesh elements near *holes* inside the Region of Interest — e.g. around a bubble, void, or cut-out region.

Refine Outer Boundary
: Subdivide mesh elements near the outer edge of the Region of Interest, where boundary effects matter most.

When either box is checked (or the Refine brush has been used on frame 1), the *Refinement Level* dropdown activates:

Level	Label	Min element size
1	Light	step/2
2	Medium	step/4
3	Heavy	step/8
4	Extra Heavy	step/16
5	Ultra	step/32

Available levels depend on subset size and subset step so the min element size stays reasonable (minimum 2 px element). A live readout below the dropdown tells you the current minimum element size.

7 Advanced parameters

The bottom collapsible section on the left sidebar holds parameters that most users leave at their defaults. Expand it only if you are debugging convergence or doing cutting-edge experiments.

7.1 Initial Guess

Controls how IC-GN gets its starting displacement for each frame. Five mutually-exclusive options:

Previous frame (default, fastest)
: Warm-start from the previous frame's converged result. Best for smooth, slow motion. Automatically selected when Tracking Mode is Accumulative.

FFT when reference frame updates
: Run an FFT cross-correlation whenever a new reference frame is adopted; warm-start within each segment. This is the default for Incremental mode.

FFT every frame (safest)
: Full FFT every frame. Eliminates warm-start error propagation across frames, but slowest. Use for unpredictable motion (oscillation, impact, random).

Reset FFT every N frames
: Full FFT every N frames, warm-start in between. Bounds the worst-case warm-start error to N frames.

Starting Points (large displacement / cracks)
: Place one or more Starting Points manually on the canvas. Each point is matched to its counterpart in the deformed frame using a focused single-point cross-correlation, then

the displacement field is propagated along mesh neighbours using an F-aware (first-order) extrapolation scheme. See Section 7.2 for the workflow.

7.2 Starting Points workflow

Use this mode when the ordinary FFT search struggles:

- **Large inter-frame displacement** (more than $\sim$ 50 pixels). FFT search grows quadratically with window size; single- point NCC scales linearly, so the whole-frame cost drops substantially in practice.
- **Discontinuous fields** (cracks, shear bands). The FFT peak picker does not know about mesh topology and can select the wrong side of a crack; propagation respects mesh connectivity and never crosses a mask hole.

Workflow:

1. Load images and set the Region of Interest normally.
2. In ADVANCED → Initial Guess, select the *Starting Points* radio button. Below it, a subpanel appears with two buttons: **Place Starting Points** and **Auto-place**.
3. Every connected mask region is overlaid in yellow on the canvas. You must place at least one Starting Point inside each yellow region; when a region has at least one point it turns green. The *Run* button stays disabled until all regions are green.
4. *Place Starting Points*: enters placement mode. Left-click inside a region to drop a point; the cursor snaps to the nearest mesh node (shown as a small circle). Right-click on a point to remove it. Press Esc to exit the mode. A banner above the canvas shows the live progress ("X / Y regions seeded").
5. *Auto-place*: scans each region and picks the node with the highest cross-correlation peak. Convenient for dense meshes; you can still remove or replace individual points afterwards.
6. The "Search Range" field in PARAMETERS is relabelled *Initial Seed Search* in this mode. It only controls the seed bootstrap search radius, which auto-expands if the peak is clipped. Other nodes use F-aware propagation, not per-node NCC.

If the pipeline cannot run your Starting Points — for instance, because one of them falls in a low-texture region or is too close to a mask boundary — a dialog explains which kind of failure occurred and suggests an action (move the point, lower the NCC threshold, fall back to FFT). See Section 14.

7.2.1 Multi-seed tolerance

When you place several Starting Points, they act as redundant anchors. In a later frame of an accumulative run, an individual seed's local template can become a poor match for the deformed image (for example because the material around it rotated or stretched significantly). pyALDIC handles this gracefully:

- Each Starting Point is bootstrapped *independently*.
- A seed whose correlation falls below the NCC threshold, or whose IC-GN solve diverges, is silently *dropped* for that frame. A short warning is logged.
- The run continues as long as every connected region still has at least one working seed among the survivors.
- If every Starting Point in a region fails, the same auto-placement algorithm used by the *Auto-place* button is invoked on that region. If it finds a viable node, the run continues with the

replacement.

- Only when no Starting Point *and* no auto-placement candidate can be found in a region does the run abort with a typed error.

Practical advice: when you expect large accumulative deformation, place several Starting Points in each region rather than one. Each acts as a backup when its neighbours degrade in late frames.

The default NCC threshold is **0.55**, chosen to tolerate the correlation decay that accumulative deformation produces while staying well above random-noise level. You can raise it for stricter matching or lower it for marginal speckle in the Advanced panel (*Seed NCC threshold*).

7.2.2 Reference-frame switches in Starting Points mode

When an incremental or hybrid frame schedule switches the reference frame mid-run, your Starting Points are *automatically warped* to the new reference's coordinate system using the cumulative displacement already computed. In normal use this is transparent: you place the points once on frame 0 and the pipeline tracks them across all switches.

If the warping fails — typically because the material point has moved outside the new reference's region of interest — the pipeline does not abort the run. Instead it calls the same 3-tier auto-placement algorithm used by *Auto-place* to drop fresh Starting Points on the new reference frame, logs a warning that names the affected frame and the number of new seeds placed, and continues with the run. The frame where this happened is marked in the frame navigator (see Section 10) so you can review the handoff after the run completes.

The run only aborts when the fallback *also* fails — that is, no node on the new reference frame meets the NCC floor. In that case the normal failure dialog appears.

7.3 Auto-expand search on clipped peaks

Checkbox, on by default. When the FFT NCC peak lands on the edge of the search window (indicating the true displacement is larger), the pipeline automatically retries with 2× search range up to six times, capped at $\min(H, W)/2$. Leave this on unless you are debugging FFT behaviour.

7.4 Mesh appearance

Cosmetic only. Choose the colour and line width of the mesh overlay drawn on the canvas (does not affect computation).

8 Canvas interactions

The centre of the window is a QGraphicsView-based canvas with layered overlays. From bottom to top:

1. Background image (reference or deformed).
2. Mesh overlay (when enabled).
3. Region of Interest outline.
4. Field overlay (post-run, colour-mapped displacement or strain).
5. Refine-brush overlay (cyan, frame 1 only).

8.1 Zoom and pan

Mouse wheel
: Zoom in / out at the cursor position.

Fit Button at the top of the canvas toolbar — zoom to fit the full image in the viewport.

100%
: Zoom to 1:1 pixel ratio.

+ / −
: Buttons for discrete zoom steps.

Middle-click drag
: Pan the view.

Left-click drag in empty area
: Pan the view (if no drawing tool is active).

8.2 Frame navigation

The frame slider at the bottom of the canvas scrubs through the sequence. The label to the left shows the current frame number and total count.

A small play/pause button next to the slider auto-advances at a configurable frame rate — useful for reviewing displacement fields after a run.

8.3 Show on deformed frame

This toggle lives in the FIELD section of the right sidebar (not VISUALIZATION, because it controls *where* the field plots, i.e. geometry, rather than how it styles).

Checked (default)
: The field overlay is plotted on the currently-selected deformed frame, with node positions warped by the cumulative displacement.

Unchecked
: The field overlay is plotted on the reference frame (frame 1) with node positions un-warped. Useful for comparing an experiment back to the starting configuration.

9 Running the DIC

9.1 The Run button

The big blue **Run DIC Analysis** button at the top of the right sidebar starts the pipeline. It is disabled until:

- At least 2 images are loaded,
- Frame 1 has a non-empty Region of Interest.

If either condition fails, clicking Run (or trying to) raises a modal dialog telling you exactly what is missing.

9.2 Cancelling

Below Run is a single red **Cancel** button (replaces the older Pause + Stop pair). It is enabled while the pipeline is running.

Cancelling stops the current run cleanly: already-computed frames are kept, no further frames are processed, and state returns to IDLE (not DONE, so the Export button stays disabled).

9.3 Progress display

Below the Cancel button:

- Thin blue progress bar (0–100 % of the full pipeline).
- Text label with the current stage (*Section 3: FFT initial search*, *Section 6: IC-GN local solve*, *S8: Frame 5/19*, . . .).
- ELAPSED / REMAINING timestamps.

9.4 Fatal errors

The console log (bottom of the right sidebar) collects every info, warning, and error message with a timestamp. *Fatal* errors (missing images, Region of Interest too small, pipeline-side exceptions) also trigger a modal dialog. The dialog shows a plain- English summary; the full traceback stays in the console log.

Warning

The console can scroll off-screen if many info messages accumulate. Fatal errors will not be missed because of the modal dialog, but warnings can be. If something looks wrong, check the console log.

9.5 Completion

When the run finishes cleanly, the Strain Post-Processing window opens automatically (see section 11). The Run button re-enables for another run; the Export button becomes active.

If the run is cancelled or fails, Strain does not open automatically and Export stays disabled.

10 Viewing results

After a run, the right sidebar populates with field-viewing controls. The canvas shows a colour-mapped overlay of the selected field on the background image.

10.1 FIELD section

Two toggle buttons (exclusive): **Disp U** and **Disp V**. Selected field is highlighted; clicking the other switches.

Below the toggles, the *Show on deformed frame* checkbox (see Section 8) controls whether the overlay plots on the deformed frame or the reference.

10.2 VISUALIZATION section

Colormap

Eight built-in maps: `jet`, `viridis`, `turbo`, `coolwarm`, `plasma`, `inferno`, `RdBu_r`, `seismic`.

RdBu_r and *seismic* are diverging (white at zero); best for displacement that crosses zero.

pyALDIC remembers the colormap choice per-field. Switching from `disp_u` to `disp_v` restores whatever map you set last time for that field.

Color Range

An explicit *Auto* / *Fixed* radio pair (replacing the former lone Auto checkbox) selects the range mode:

Auto (default)
: Range is set from the 2–98 percentile of visible node values (excludes extreme outliers but uses real data), rescaled every frame.

Fixed
: Type *Min* and *Max* explicitly; the bounds are kept for every frame. Useful for reporting (all frames share one range) or when you know the physically meaningful bounds.

The same Auto / Fixed pair appears in the Strain window and in the Export dialog, so the range mode is always an explicit choice.

Note

Numeric input fields accept both decimal separators: typing `0.07` or `0,07` gives the same value regardless of your operating-system locale.

Opacity

Slider (0–100%). Default 70%. Adjusts the alpha blending between the field overlay and the background image.

10.3 PHYSICAL UNITS section

A checkbox plus two fields:

Pixel Size
: Numeric value, dropdown unit (nm, µm, mm, cm, m, inch). Converts raw pixel displacements to physical length on all displacement-family fields.

Frame Rate
: Used to convert inter-frame velocity (ΔU/ frame) to physical speed (length / s).

Strain fields are dimensionless and unaffected by these settings.

10.4 Console log

Scrollable read-only widget at the bottom of the right sidebar. Colour-coded by severity:

Level	Colour
info	blue / white
warning	yellow
error	red
success	green

Every line is prefixed with a `HH:MM:SS` timestamp.

11 Strain post-processing

The Strain window is a separate top-level window that opens automatically when a Run completes. It computes strain from the stored displacement field, with its own parameter panel and its own visualization state (independent of the main window).

You can also open it manually via the **Open Strain Window** button in the right sidebar.

The right column of the strain window is organised as a scrollable stack of collapsible sections. *Strain Parameters*, *Field*, and *Visualization* are expanded by default; *Physical Units* and *Log* start collapsed — click their header (triangle arrow) to expand. The window opens fitted to the available screen area (it is never larger than your desktop, even on small laptops); if the content does not fit, the right column scrolls.

11.1 Strain Parameters panel

Three editors from top to bottom:

Method

Plane fitting (default)
: Weighted moving least-squares fit of a first-order polynomial to the displacement field inside a virtual strain gauge (VSG). Gaussian weights with the VSG radius as σ. Robust, smooths local noise.

FEM nodal
: Strain from shape-function derivatives of the FEM mesh. Higher spatial resolution but more sensitive to per-node noise.

VSG size (plane fitting only)

Virtual Strain Gauge diameter in pixels. Odd integer. Default 41. Larger = more smoothing. The field reports radius internally as $(VSG - 1)/2$.

A live *Strain window* $\approx N \times N$ *nodes* readout below the control translates the pixel VSG into the number of mesh data points the plane fit spans on a uniform mesh: with node spacing (subset step) s and radius $r = (VSG-1)/2$, a node's k-th axial neighbour lies $k\,s$ away, so $N = 2\lfloor r/s \rfloor + 1$ nodes fall on each axis — exactly the set the radius search selects. The readout updates live as the VSG or the DIC step changes. (On a refined mesh the node count varies locally, so the readout reflects the uniform-mesh case.)

Trim low-confidence edges (plane fitting only)

At nodes near the ROI boundary (or the edge of an internal hole / crack), the VSG window crosses the boundary, so the local plane fit only sees points on one side and becomes biased and unreliable. This control hides those edge nodes from the displayed and exported strain.

The coefficient α (slider 0–1, default 0.70) sets how wide a boundary band to drop: a node is trimmed when its distance to the boundary is below $\alpha\times$ VSG radius.

$\alpha = 0$ No trimming — every node is reported (legacy behaviour).

$\alpha = 0.70$ **(default)**
: Trims the band where the plane-fit error is elevated; calibrated so the kept region matches where the error returns to the interior baseline.

$\alpha = 1.0$
: Strictest — trims any node whose VSG window touches the boundary.

A live *Trimmed: N nodes (M%)* readout below the slider shows how many nodes the current setting removes. By default the trimmed nodes appear as a transparent band just inside the ROI edge, on the canvas and in exports alike (but see *Fill trimmed edges* below). (FEM nodal strain is computed from element connectivity rather than a radius, so this control does not apply to it.)

Warning

If $\alpha\times$ VSG radius exceeds the Region's half-thickness, *every* node is trimmed and both the strain map and the export come out entirely blank / NaN (the readout reads 100%). Reduce the VSG size, lower α, or switch to FEM nodal — see Section 14.

Fill trimmed edges (display only)

A checkbox in the strain visualization controls (off by default). When enabled, the transparent trim band is filled back in from the reliable interior nodes — interpolated where possible and *extrapolated* out to the full ROI edge where trimming the outer ring shrank the interpolation hull — for the on-canvas view and for exported *images and animations*. It is a display convenience only: the low-confidence boundary nodes are never shown (the fill comes entirely from reliable interior nodes), and exported *data* files (`NPZ` / `MAT` / `CSV`) always keep the trimmed edge as `NaN` regardless of this toggle. Leave it off when you need the trim boundary to remain visible; note the extrapolated edge is an estimate, so treat those outermost pixels as indicative only.

The fill also recovers the band trimmed along a crack or hole *inner* edge — but each cell is filled only from nodes on *its own side* of the crack (the straight line to the source node may not cross the crack). The two crack faces are therefore filled independently and the crack itself stays a sharp transparent line; the fill never bridges it.

Crack-aware rendering

Where the ROI contains a crack or an internal hole, the field overlay no longer interpolates across it: rendering respects the same discontinuity the mesh does, so the two faces of a crack are drawn independently instead of being smeared into a smooth ramp. This is automatic (no control) and applies to both the displacement and strain overlays, on the canvas and in exported images and animations.

Reference vs. deformed geometry

Strain is total-Lagrangian, so it is always computed on the frame-0 reference mesh, but the *geometry* used to display it follows the *Show on deformed frame* toggle:

Not deformed (reference view)
: The strain is drawn on the reference frame using *frame-0* geometry — the same first-frame ROI mask the main window's displacement overlay uses. A crack that has grown by the current frame is *not* carved into the reference view (matching the displacement).

Show on deformed frame
: The strain is drawn on the current frame using that frame's geometry: the current crack is carved in, edge-trim follows it, and *Fill trimmed edges* recovers it face-by-face as above.

So toggling the view switches the whole geometry (mask, edge-trim, fill, crack) between first-frame and current-frame consistently.

Strain field smoothing

Gaussian smoothing of the strain tensor field *after* computation (not a displacement smoother). Dropdown with four presets and a warning glyph on the aggressive one:

Preset	σ relative to step	Behaviour
Off	0	No smoothing
Light	0.5	Subtle, preserves fine features
Medium	1.0	Balanced (recommended for noisy data)
Strong ▲	2.0	Aggressive, may blur real gradients

Here "step" is the DIC node spacing (the value you set as *Subset Step* in the main window). Smoothing smaller than 0.5× step has essentially no effect because the Gaussian kernel cannot reach any neighbour node; 2× step is the upper bound of "useful" smoothing.

Strain type

The strain measure the output fields report:

Infinitesimal (default)
$\varepsilon = \frac{1}{2}(\nabla u + \nabla u^T)$. Accurate for small deformation only. *Not* zero under finite rigid rotation — it reports $\cos\theta - 1$ as false strain.

Eulerian–Almansi
$e = \frac{1}{2}(I - F^{-T}F^{-1})$. Reported in the current (deformed) configuration.

Green–Lagrangian
$E = \frac{1}{2}(F^T F - I)$. Reported in the reference configuration. **Exactly zero under rigid rotation.** Use this for any experiment with rotation $> 2°$.

11.2 Compute Strain button

Green primary button below the parameters. Click to recompute strain with the current panel settings. Runs in a background thread; a progress bar shows per-frame progress.

If you haven't changed any parameter and the strain is already computed, the stale warning label stays empty. If you change anything, a yellow *Params changed — click Compute Strain* hint appears.

11.3 Strain field selector

Below the parameters, a radio-style selector with all available fields:

- **Displacement** (shared with main window): `disp_u`, `disp_v`, `disp_magnitude`, `velocity`
- **Strain components**: `strain_exx`, `strain_eyy`, `strain_exy`
- **Derived strains**: `strain_principal_max`, `strain_principal_min`, `strain_maxshear`, `strain_von_mises` `strain_rotation` (polar-decomposition rotation angle in degrees)

Visualization controls (colormap, range, opacity) and physical-unit settings mirror the main window's but are *independent*: the strain window remembers its own colormap and range choices per field.

11.4 Strain vs main window

	Main window	Strain window
Shows	Displacement field	Strain fields (plus disp)
State	Shared AppState	Private viz state
Export button	Export Results	Export Results (same dialog)
Frame navigator	Shared	Private

12 Exporting

Both the main window and the Strain window have an **Export Results** button. Both open the *same* dialog. The difference is only which visualization preset (colormap, range, deformed toggle) seeds the dialog — whichever window you opened it from.

12.1 The Export dialog

Five tabs at the top of the dialog:

Data tab

Check boxes for numeric-data formats:

- **NPZ** — NumPy compressed archive; preferred for Python downstream.
- **MAT** — MATLAB MAT-file (scipy v5).
- **CSV** — one row per node per frame; plain text, large.

Plus a multi-select list of which fields to include (displacement, strain components, derived strains, rotation).

Variable structure

All three data formats expose the *same* variable names, so a downstream script reads identically regardless of format:

Variable	Shape	Meaning
`coordinates`	$(N, 2)$	reference node x, y (pixels)
`disp_u`, `disp_v`	(N, T)	accumulated displacement
`disp_magnitude`	(N, T)	$\sqrt{u^2 + v^2}$
`strain_*`	(N, T)	strain components / derived

N is the node count and T the number of frames. Strain columns are aligned to the displacement frames — a frame with no computed strain is left as NaN — so every matrix shares the same column count. In CSV the same fields are columns (`node_id, x_px, y_px, disp_u, ...`), one row per node, one file per frame.

Note

Displacement and strain are both reported in **world display coordinates**: the y-axis points up (opposite the image row direction), and values scale by the pixel size when physical units are enabled. This matches the on-screen fields, so `disp_v` and the shear/strain signs stay consistent within one file. The MAT file also stores `n_frames`, `n_nodes`, and a `DICpara` struct of the run parameters; a complete parameter dump is always written separately as `<prefix>_parameters_<timestamp>.json`.

Images tab

Per-field list with Export / Colormap / Auto / Min / Max / opacity (α) columns for rendering each field as JPEG / PNG / TIFF. One file per frame per enabled field. The same appearance settings can also be edited with a live preview on the Preview & Colorbar tab, which stays two-way synced with this list.

Tip

Include colorbar is ON by default. Images without a colorbar show coloured patches but no scale, which is almost never useful for quantitative reports. Uncheck it only if you plan to composite a manually-placed colorbar in Illustrator / Inkscape.

Adjustable settings:

Format
: JPEG (default) / PNG / TIFF. JPEG at the default quality is visually indistinguishable for field overlays and far smaller; pick PNG or TIFF when you need lossless output.

Resolution (long edge)
: Caps the exported image's long edge (the larger of width and height; aspect ratio is kept): 512 / 768 / 1024 (default) / 1536 / 2048 px, or *Full resolution* for native size. Field detail is bounded by the mesh, so a cap is near-lossless but much smaller on disk and faster to encode.

JPEG quality
: 60–100, default 92. Ignored for PNG / TIFF.

DPI
: 72–600, default 150 (metadata for print sizing).

Render as
: *Original (frame 1 background)*: field drawn at the undeformed node positions on the reference image. *Deformed (current frame background)*: field drawn at the displaced node positions on each frame's own image.

Note

Images render directly at the output resolution with a lookup-table colormap, so exports are roughly 5× faster and 35× smaller than in pyALDIC 0.5 at the default settings — a 90-frame 4K batch drops from 3.3 min / 2.8 GB to about 40 s / 80 MB.

Animation tab

Render an MP4 / GIF for each selected field, sweeping through frames. Frames are encoded one at a time (streaming), so even long 4K sequences export without a memory spike. Colorbar on by default here too. Settings:

Format
MP4 / GIF.

Resolution (long edge)
Same presets as the Images tab (512–2048 px or full resolution). Strongly recommended for GIF, whose file size explodes at native resolution.

FPS Playback rate; usually match the frame rate of the experiment.

Frame step
Export every Nth frame (1 = every frame). Higher is faster and produces smaller files but looks choppier. Playback duration is preserved: FPS refers to the pre-decimation timeline.

Warning

MP4 export requires `ffmpeg` on your system PATH. If ffmpeg is missing, the dialog will switch silently to GIF and log a warning.

Report tab

Generates a multi-page PDF summarizing the run: parameters, key fields at mid and last frame, RMSE (if ground truth is available), and optional thumbnail animation.

Preview & Colorbar tab

A WYSIWYG preview of one exported frame, rendered through the *real* export path — what you see here is exactly what Export Images and Export Animation will write. Pick the field and scrub the frame slider under the preview; it re-renders as you change any setting.

Two control groups on the right:

FIELD APPEARANCE
Colormap, Auto / Fixed range with Min / Max, and opacity for the previewed field. These are *two-way synced* with the matching row on the Images tab: editing here updates that row and vice versa. *Apply to all fields* copies the colormap, opacity, and range mode to every enabled field (each field keeps its own Min / Max).

COLORBAR STYLE
Position (Right / Left / Top / Bottom), *Font size* (6–32 pt), *Font family* (sans-serif / serif / monospace), *Bar thickness* (as a fraction of the image), *Background* (Black / White), and an outward *Margin* (blank border around the exported content, as a fraction of the long edge) with its own *Margin color*. These apply to image and animation exports alike.

When physical units are enabled at the top of the dialog, the colorbar label and tick values use them (e.g. mm instead of px), both in the preview and in the exported files.

12.2 Destination folder

At the top of the dialog. Defaults to `<image_folder>/al-dic-export/<timestamp>/`. Click *Browse* to pick a different location. Exports are written inside that folder with a user-supplied prefix (default `result`).

12.3 Running an export

Click the primary **Export** button at the bottom. Progress shows per-file (data) and per-frame (images / animations). The Cancel button stops cleanly; already-written files are kept.

When the export completes, a green success message tells you how many files were written and their location. The dialog stays open so you can export more formats; click Close when done.

13 Sessions — save and resume

pyALDIC can save your *entire* project — image folder reference, parameters, Regions of Interest, view state, and (optionally) the computed results — into a single `.aldic` file. Opening the session later lands you back on the exact page you left: same frame, same field, same colormap and colour ranges, with no recompute. Hours of computation survive closing the GUI.

13.1 File format

- Extension: `.aldic` — a single-file bundle containing a human-readable JSON configuration (`session.json`) plus, when results are included, a compressed NumPy archive (`results.npz`).
- Single-file portable: Regions of Interest are stored inline as base64-encoded PNG. Result arrays are deduplicated by content (per-frame meshes that are identical across an accumulative run are stored once) and written without Python pickling, so opening a bundle from an untrusted source can never execute code.
- Versioned: the `schema_version` key protects against silent misinterpretation of newer schemas. Legacy `.aldic.json` sessions from pyALDIC 0.5 and earlier still open (see below).

13.2 What is saved

- Image folder path and the ordered list of filenames.
- Every per-frame Region of Interest mask.
- Core DIC parameters: subset size, step, search range, tracking mode, reference-update policy, solver choice, refinement settings, initial-guess mode.
- Physical units (pixel size, unit, frame rate).
- View state: current frame, selected field, deformed / reference toggle, overlay opacity, mesh appearance, and each field's colormap, range mode, and bounds.
- **Computed results** (optional): displacement and strain fields plus the mesh, byte-faithfully — reopening the session needs no recompute.

13.3 What is not saved

- The images themselves — the bundle stores the folder path and filenames, never pixel data. Keep the image folder around (or re-select it after moving).
- Run state / progress / elapsed time.
- Starting Points (the seed markers of the *Starting Points* init-guess mode). Seeds reference mesh node indices, which change with the Region of Interest and parameters — re-place or Auto-place them after opening a session.

13.4 Menu

File → Save Session...
(`Ctrl+S`) Opens a file dialog. Extension `.aldic`, appended automatically. Overwrites if the file exists.

File → Open Session...
(`Ctrl+O`) Opens a file dialog. Filter: `pyALDIC Session (*.aldic *.aldic.json)`.

File → Associate .aldic files with pyALDIC...
Windows only. Registers the `.aldic` extension for the current user (no administrator rights needed) so double-clicking a session file in Explorer launches pyALDIC and opens it directly. The command line works the same way: `al-dic path/to/run.aldic`.

13.5 Including results

If results exist when you save, a prompt asks whether to include the computed results in the bundle and shows an estimate of their uncompressed size:

Yes (default)
Writes the full results. Reopening the session restores every displacement and strain field without recomputing — for long runs this is the whole point.

No Writes a small configuration-only file (parameters, Regions of Interest, view state). Ideal for sharing a setup with a colleague.

Saving and loading run on a background thread behind a progress dialog, so multi-gigabyte bundles never freeze the GUI. The progress bar tracks the actual bytes written, array by array.

13.6 Opening a session whose images moved

If the image folder path in the session no longer exists, pyALDIC does *not* fail the load. Instead it:

1. Restores parameters and Regions of Interest.
2. Logs a warning: "Image folder ... no longer exists; parameters and Regions of Interest were restored but images must be re-selected manually."
3. Leaves the image list empty.

You can then drop a new folder of the same images and the Regions of Interest will still be aligned.

Tip

This behaviour is deliberate so you can share a `.aldic` bundle with a colleague whose images live at a different path. They re-drop the images; all parameters and Regions of Interest come along.

13.7 Corrupt or future-version files

A corrupt bundle (unreadable zip), malformed JSON, or an unknown `schema_version` raise a modal error dialog. The fatal- error path is used so you cannot miss the failure. The application keeps whatever state it had before the attempted load; nothing is silently replaced.

13.8 Legacy sessions from pyALDIC 0.5 and earlier

Older versions saved sessions as a plain JSON file with the extension `.aldic.json`. These still open through File → Open Session: the loader sniffs the file content (zip bundle vs. plain JSON) rather than trusting the extension. Legacy files contain no results or view state, so those parts simply restore to defaults. Saving again writes the current `.aldic` bundle format.

14 Troubleshooting

14.1 "Cannot start analysis"

A modal dialog with one of these messages:

Need at least 2 images
Drop an image folder into the Images panel.

Define a Region of Interest first
Use the Region of Interest toolbar to draw or import one on frame 1.

Region of Interest is empty
You drew something, then clicked Cut over the whole region. Click Clear or redraw.

Region of Interest too small
The bounding box is smaller than 2 × Subset Step. Enlarge the Region or reduce Subset Step.

14.2 "DIC analysis failed"

Shows an exception type and message. The full traceback stays in the console log — scroll up to find it. Common causes:

- Images of different sizes (should be caught at load; if you see it at Run, report a bug).
- Extremely low subset correlation (black-on-black image or no speckle). Check your input data first.

14.3 Starting Points errors

Specific to the *Starting Points* init-guess mode. Each error dialog carries a suggested action; the summary here explains *why* each situation happens.

Starting Point match quality too low
The single-point cross-correlation between reference and deformed frame at that point scored below the NCC threshold (default 0.70). The local texture is too uniform to match reliably. Move the point into a better-speckled area, or lower the threshold if you know the images are noisy.

Starting Point local solver did not converge
The IC-GN iteration at the point hit the max iteration count without converging. Most often caused by a point too close to a mask boundary or an oversized subset window that samples outside the mask. Move the point further inside the region or reduce subset size.

Starting Point flagged as unreliable after run
The Starting Point converged but its iteration count was a statistical outlier compared to other nodes, so the propagation result is suspect. Pick a cleaner region or add redundant points.

Could not carry Starting Points to the new reference
On a reference-frame switch (custom or every-N-frames schedules), the stored Starting Point warped outside the new reference's ROI *and* the auto-placement fallback also found no viable node on the new reference frame (every candidate fell below the NCC floor). The pipeline normally recovers from a warp failure by auto-placing fresh Starting Points and continuing — this error only appears when the new reference itself is unusable. Place the original

points well inside the ROI interior, widen the new ROI, or improve the reference-frame image quality.

Region without a Starting Point

At least one connected mask region is not represented. The propagation cannot cross mask boundaries — every region needs a point. Either add one per yellow region, or unify the mask so only one connected region remains.

14.4 The very first analysis takes far longer than the rest

Expected, once per installation. The solver is compiled to machine code on first use; on a fast workstation that is around half a minute, more on a laptop. pyALDIC starts the compilation in the background shortly after the window opens, so it usually finishes while you are still loading images — watch for *"Compute kernels ready"* in the console log.

Warning

If the interface seems frozen during that first run, wait rather than killing the process. The compiled result is only written to the cache once compilation finishes, so a forced quit means the next launch starts over.

14.5 Slow run

- Subset Step too small: halve the step → 4× node count → 4× runtime. Start with step = 16 for large images.
- AL-DIC solver: ∼ 3× slower than Local DIC. Preview with Local DIC, switch to AL-DIC for the final run.
- Search Range much larger than actual motion: every FFT call costs proportional to (search)^2. Set it just above your expected inter-frame motion.

14.6 Poor results in specific regions

- Noisy or fuzzy edges of the Region of Interest: enable *Refine Outer Boundary* to densify mesh near the edge.
- Hole boundaries (e.g. bubble edges): enable *Refine Inner Boundary*.
- Whole regions that simply failed to converge: check the console log for "n_bad_pts" warnings. Usually means Search Range was too small or the speckle was washed out there.

14.7 Strain map is blank or export is all NaN

The displacement fields look fine, but after *Compute Strain* the strain overlay shows nothing (only the background image) and every exported strain value (NPZ / CSV / MAT / PNG) is NaN.

With the default **Plane fitting** method this means the edge-trim removed *every* node. A node is dropped when its Virtual Strain Gauge (VSG) window comes within $\alpha \times$ VSG radius of the ROI, hole, or image boundary (see Section 11). With the default VSG size 41 px (radius 20 px) and $\alpha = 0.70$ the trim band is 14 px, so a Region thinner than about 28 px — or a normal specimen with the VSG size raised too high — has *all* of its nodes trimmed. The live *Trimmed: N nodes (M%)* readout reads 100% in this case.

Fixes (any one):

- Reduce **VSG size** so that $\alpha \times (\text{VSG} - 1)/2$ stays below the Region's half-thickness.

- Lower **Trim low-confidence edges** (α) toward 0; setting it to 0 disables trimming entirely.
- Switch **Method** to **FEM nodal**, which uses no VSG window and no edge trim.

The opposite extreme — a VSG size *below* the DIC node spacing (Subset Step) — fails differently: the plane fit then has too few neighbours, so no strain is produced and the reason is written to the *Log* section (expand it to read the message). Keep the VSG radius $\geq$ Subset Step.

14.8 Large rotation gives crazy strain values

You are probably using *Infinitesimal* strain with a rotation $> 2°$. Infinitesimal strain interprets rigid rotation as strain of magnitude $\cos\theta - 1$ (about -0.13 at 30°). Switch the Strain window's *Strain type* to **Green–Lagrangian**, which is exactly zero under any rigid rotation.

14.9 Refine brush grayed out

Brush is gated to frame 1 only. Click into the image list or use the frame slider to return to frame 1. The button re-enables automatically.

14.10 Session file won't open

The fatal-error dialog shows the specific reason:

- File corrupted or hand-edited wrong: legacy `.aldic.json` files are plain JSON and can be checked with a JSON linter; `.aldic` bundles are zip archives and should not be edited by hand.
- `Unsupported schema version`: the file was saved by a newer pyALDIC version than the one you are running. Upgrade pyALDIC.
- `Image folder ... no longer exists`: this is a warning, not a failure. Parameters and Regions of Interest still load; re-select the image folder manually.

14.11 Windows bundle: "Windows protected your PC"

SmartScreen showing an unrecognised-publisher notice on first launch. The bundle is not code-signed, so Windows has no reputation record for it. Choose **More info**, then **Run anyway**. If your antivirus quarantines the folder instead, restore it and add an exclusion — self-contained Python bundles are a common false positive.

14.12 Windows bundle: nothing happens on double-click

- Check the log at `%LOCALAPPDATA%\pyALDIC\logs\pyALDIC.log`. Every startup writes to it, and an unhandled error is recorded there in full.
- Run `pyALDIC-console.exe` from the same folder. It is the identical application with a console attached, so any error that has nowhere to go in the windowed build appears there.
- Make sure the folder was unzipped, not merely opened. Windows lets you browse inside a `.zip` and run an `.exe` from there; it will not find the files it needs.
- Keep the path short. The bundle needs about 100 characters of its own, so unzipping into an already deeply nested folder can exceed the Windows path limit.